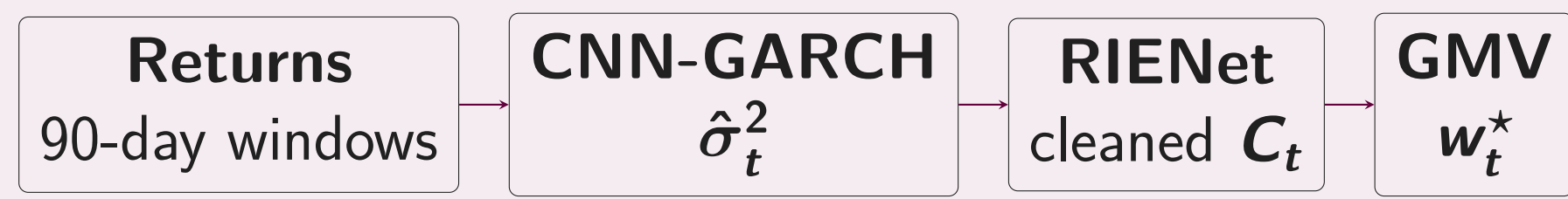


1. Motivation, Research Problem, and Contributions

Motivation Global Minimum Variance (GMV) portfolios are highly sensitive to estimation noise: unstable marginal volatilities and noisy rolling correlations create fragile inverse-covariance weights.

Research problem Can we improve out-of-sample GMV stability by learning a better covariance-estimation pipeline while keeping the downstream optimizer fixed?



Main contributions

- ▶ a **hybrid CNN-GARCH** model for marginal volatility forecasting,
- ▶ a **stable** (ω , C , ρ) parameterization,
- ▶ a **RIENet** layer for rolling correlation cleaning,
- ▶ a fair GMV comparison where only the covariance estimate changes.

Evaluation logic

- ▶ first reduce **portfolio volatility**,
- ▶ then compare **Sharpe ratio**.

2. Data Cleaning and Experimental Setup

Dataset: WIKI adjusted prices from 2000-01-04 to 2018-03-27.

Cleaning rules

- ▶ remove zero-volume observations,
- ▶ compute adjusted-close returns,
- ▶ keep only series complete over the training timeline,
- ▶ exclude low-price stocks using a **\$3** threshold on the first test day,
- ▶ exclude assets with more than 20 consecutive zero returns.

Final setup

- ▶ 1338 cleaned stocks in the full universe,
- ▶ 50 stocks in the end-to-end GMV poster experiment,
- ▶ 70% train / 30% test split,
- ▶ train-only normalization to avoid leakage.

Why the cleaning matters

- ▶ low-price and stale-trading stocks create unstable returns,
- ▶ incomplete histories create biased covariance estimates,
- ▶ the filtered universe is designed for robust out-of-sample testing.

3. CNN-GARCH Architecture

Input: one 90-day return window per stock.

Architecture

- ▶ 5 Conv1D layers,
- ▶ kernel size **3**,
- ▶ same padding,
- ▶ dilations **1, 2, 4, 8, 16**,
- ▶ GroupNorm + LeakyReLU,
- ▶ global average pooling + dense prediction head.

Effective receptive field

$$RF = 1 + (k - 1) \sum_{i=0}^4 2^i = 1 + 2(1 + 2 + 4 + 8 + 16) = 63.$$

So the convolutional stack directly captures **63 days** of temporal context, while the pooled representation still summarizes the full **90-day window**.



4. Stable GARCH Parameterization

Instead of predicting α and β directly, the network predicts:

$$\omega = \text{softplus}(h_\omega), \quad C = 0.99 \sigma(h_C), \quad \rho = \sigma(h_\rho).$$

Then:

$$\alpha = C\rho, \quad \beta = C(1 - \rho).$$

Why this is useful

- ▶ C is constrained between 0 and 1,
- ▶ ρ is constrained between 0 and 1,
- ▶ therefore $\alpha, \beta \in (0, 1)$,
- ▶ and $\alpha + \beta = C < 1$, which stabilizes persistence.

Differentiable GARCH layer

$$\hat{\sigma}_{t+1}^2 = \omega + \alpha r_t^2 + \beta \hat{\sigma}_t^2.$$

The CNN learns the parameters, and the GARCH recursion computes the next variance.

5. Training Strategy

Stage 1: synthetic pretraining. The model first learns generic GARCH dynamics on many simulated GARCH(1,1) series.

Stage 2: fine-tuning on real equities. It is then adapted to filtered WIKI returns with train-only normalization, zero-aware sampling, an r^2 floor, and validation-based checkpoint selection.

Loss

$$\mathcal{L} = \text{MSE}(\log \sigma_{\text{true}}^2, \log \hat{\sigma}^2).$$

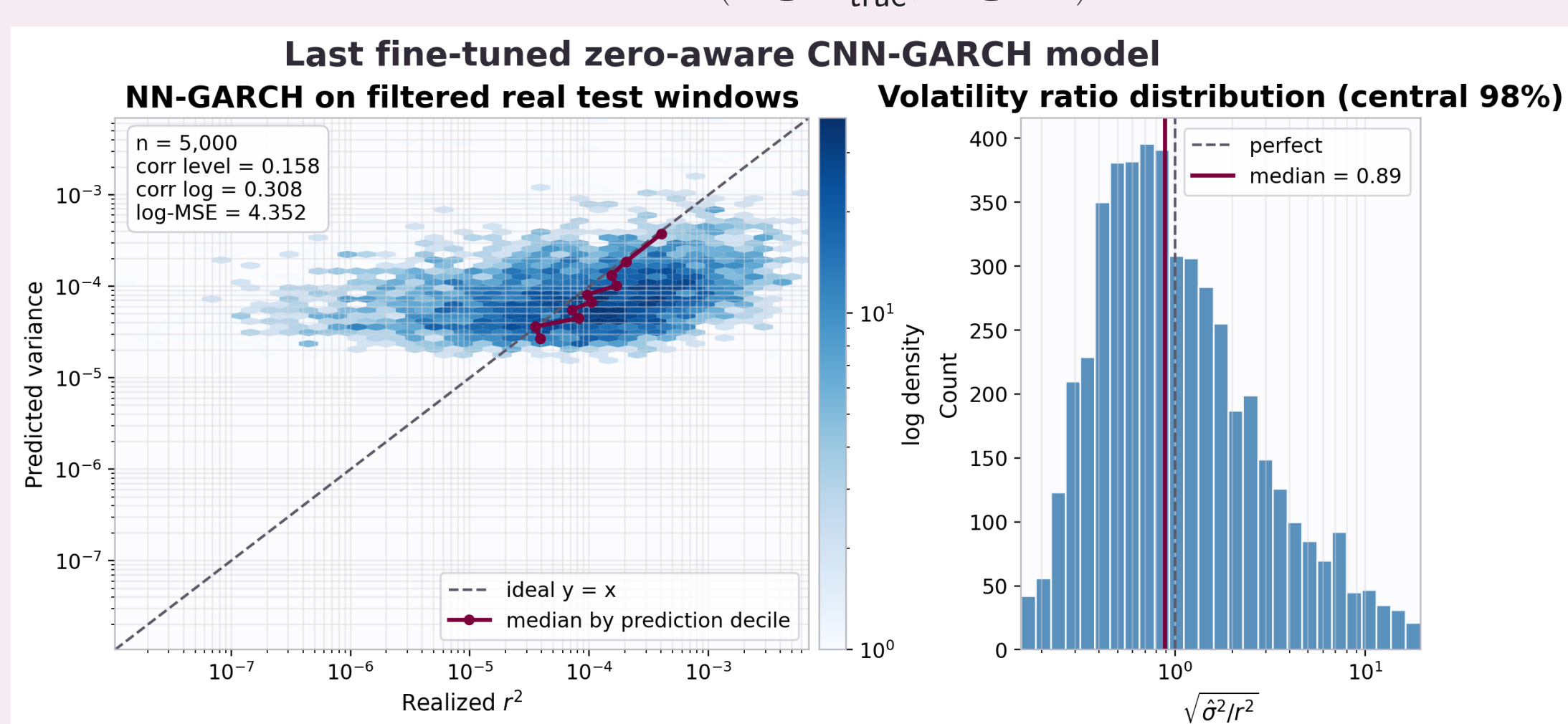


Figure 1. Real filtered OOS windows: prediction density and volatility-ratio distribution.

6. RIENet Correlation Cleaning

Motivation Rolling sample correlations are noisy. That noise propagates directly to Σ_t^{-1} and makes GMV weights unstable.

RIENet layer explanation

1. eigen-decompose the rolling correlation matrix,
2. enrich the eigenvalue sequence with dimension-aware information such as $q = T/N$ and lookback length,
3. clean the spectrum in inverse-eigenvalue space with a recurrent transform,
4. reconstruct a cleaned correlation matrix.

Interpretation

- ▶ CNN-GARCH estimates the **marginal variances**,
- ▶ RIENet cleans the **dependence structure**,
- ▶ together they produce a more stable covariance matrix.

$$\Sigma_t = D_t C_t D_t, \quad w_t^* = \frac{\Sigma_t^{-1} \mathbf{1}}{\mathbf{1}^\top \Sigma_t^{-1} \mathbf{1}}.$$

7. Related Work, GMV Construction, and Baselines

Related work Classical GMV relies on rolling sample covariance and GARCH-style volatility models. Covariance cleaning literature uses shrinkage and random-matrix denoising, and recent end-to-end portfolio optimization papers motivate learning better inputs to the optimizer instead of changing the allocation rule.

Covariance model

$$\Sigma_t = D_t C_t D_t$$

with $D_t = \text{diag}(\hat{\sigma}_{1,t}, \dots, \hat{\sigma}_{N,t})$.

Unconstrained GMV

$$w_t^* = \frac{\Sigma_t^{-1} \mathbf{1}}{\mathbf{1}^\top \Sigma_t^{-1} \mathbf{1}}.$$

Methods compared

- ▶ **GMV-GARCH:** GARCH volatilities + raw rolling correlation,
- ▶ **GMV-NN:** CNN-GARCH volatilities + raw rolling correlation,
- ▶ **GMV-NN-Clean:** CNN-GARCH volatilities + RIENet-cleaned correlation.

Fair comparison All three methods use the same stocks, the same lookback window, and the same GMV formula. Only the covariance estimate changes.

8. Main Out-of-Sample Results

50-stock unconstrained GMV experiment, 2012–2018

Method	Ann. Ret.	Ann. Vol.	Sharpe
GMV-GARCH	17.39%	13.95%	1.2468
GMV-NN	20.43%	14.40%	1.4190
GMV-NN-Clean	17.53%	12.75%	1.3755

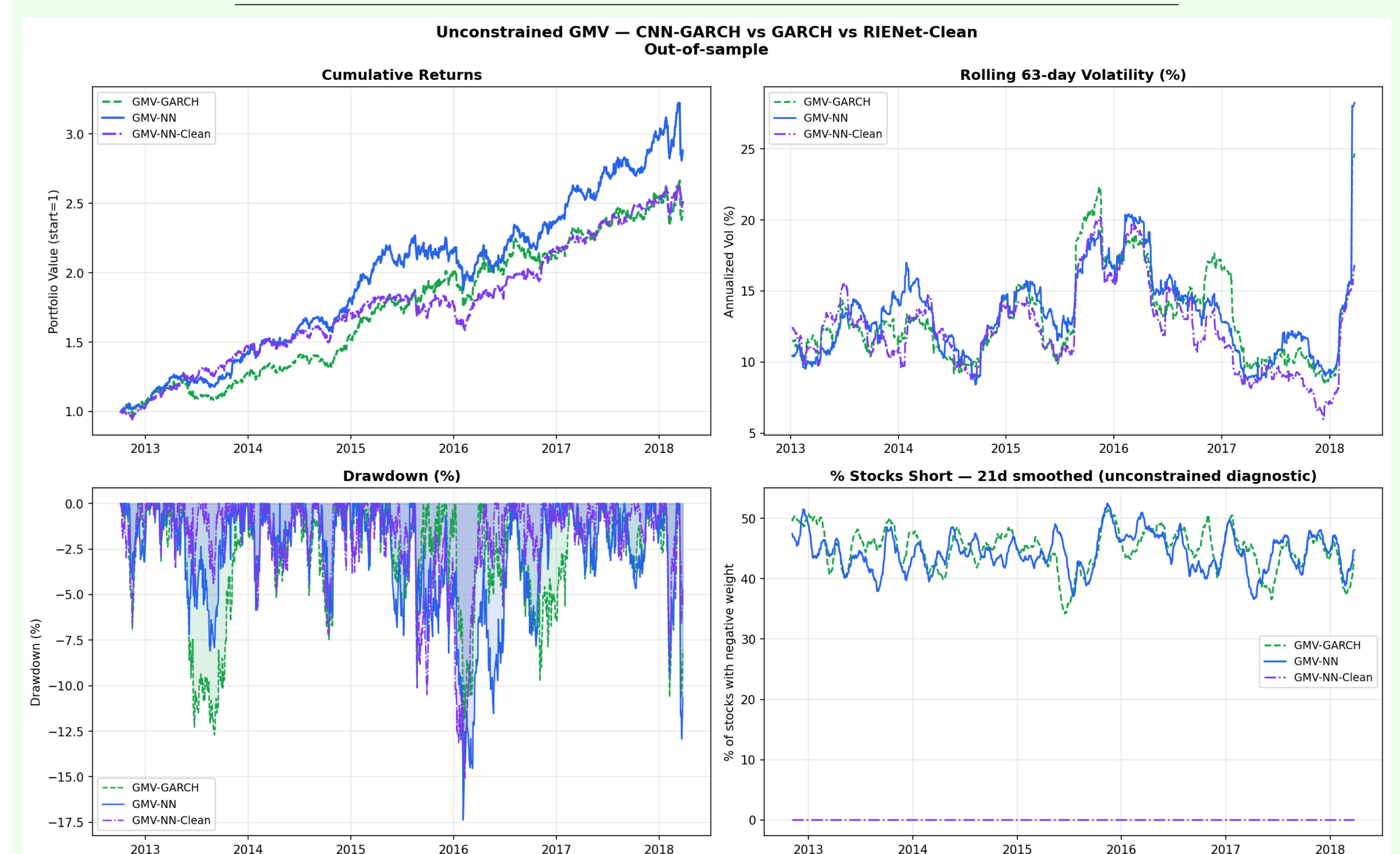


Figure 2. Cumulative return, rolling volatility, drawdown, and short-weight diagnostic.

9. Conclusion, Discussion, and Perspectives

Discussion

- ▶ GMV-NN-Clean gives the **lowest out-of-sample volatility**.
- ▶ GMV-NN gives the **highest Sharpe ratio**.
- ▶ RIENet cleaning regularizes spectral inversion and reduces fragile allocations.

Robustness On 200 random 30-stock subsets, NN-GARCH also improves average volatility and Sharpe, and Wilcoxon signed-rank tests are significant for both metrics.

Perspectives

- ▶ scale beyond 50 assets and include transaction costs or long-only constraints,
- ▶ jointly train volatility and correlation modules end-to-end,
- ▶ study turnover regularization and alternative risk objectives.

Take-away

$$\hat{\sigma}^2 \uparrow \text{quality} + C_t \uparrow \text{quality} \implies \Sigma_t \uparrow \text{quality} \implies \text{more stable GMV portfolios.}$$

